MDP를 모를 때 밸류 평가하기

- 모델 프리(model-free) 상황: 실제로 어떤 상태로 이동하게될지 확률분포도 전혀 모르는 상황
- 모델(model): 강화학습에서 환경의 모델
    - 에이전트 입장에선 여러가지 계획(planning)을 세워볼 수 있음
- 즉, MDP를 모른다 = 모델을 모른다 = 모델 프리다

5.1. 몬테 카를로 학습



| 그림 5-1 | 앞면이 나올 확률을 모르는 동전

- 위 동전은 무게중심이 한 쪽에 치우쳐 앞면이 나올 확률이 50%가 아님
- 이걸 10번 던져서 앞면이 3번 나왔으므로 30%로 가늠
    - ->기댓값: $0.3 * 100 + 0.7 * 0 = 30 \frac{3*100}{10} = 30$
- 몬테 카를로 방법론(modte Carlo Method): 여러번 던질수록 그 값이 점점 정확해진다는 이론

가치함수에서의 몬테 카를로

- 가치함수: $v_\pi(s_t) = E_\pi[G_t]$
    - 즉 가치함수의 정의는 리턴의 기댓값
- 리턴을 여러번 계산해서 그 평균을 내면 실제 가치에 수렴할 것
    - 대수의 법칙(law of large numbers)

몬테 카를로 학습 알고리즘

s_0	s_1	s_2	s_3
s_4	s_5	s_6	s_7
s_8	s_9	s_{10}	s_{11}
s_{12}	s_{13}	s_{14}	종료

→

(0,0)	(0,0)	(0,0)	(0,0)
(0,0)	(0,0)	(0,0)	(0,0)
(0,0)	(0,0)	(0,0)	(0,0)
(0,0)	(0,0)	(0,0)	(0,0)

| 그림 5-4 | 초기화된 테이블

- 에이전트는 4방향 랜덤으로 이동
    - 이번에는 보상함수 r_s^a와 전이확률 $P_{ss'}^a$를 모름
- 경험을 기반으로 배워야 함
    - 에이전트가 4방향 랜덤으로 이곳 저곳을 돌아다니며 보상과 상태 전이에 대한 경험을 쌓을 것임

몬테카를로 과정

1. 테이블 초기화

s_0	s_1	s_2	s_3
s_4	s_5	s_6	s_7
s_8	s_9	s_{10}	s_{11}
s_{12}	s_{13}	s_{14}	종료

→

(0,0)	(0,0)	(0,0)	(0,0)
(0,0)	(0,0)	(0,0)	(0,0)
(0,0)	(0,0)	(0,0)	(0,0)
(0,0)	(0,0)	(0,0)	(0,0)

| 그림 5-4 | 초기화된 테이블

- 각 상태별로 $N(s)$와 $V(s)$ 2개의 값 필요
    - $N(s)$: s의 방문 횟수
    - $V(s)$: 해당 상태에서 경험했던 리턴의 총 합

2. 경험 쌓기

- 정책 π를 이용해 움직이는 에이전트를 그리드 월드의 출발점 s_0에다 가져다 놓음
- 에피소드

$$s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T$$

- 각 상태에 대한 리턴

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \cdots + \gamma^{T-1} r_{T-1}$$

$$G_1 = r_1 + \gamma r_2 + +\gamma^2 r_3 \dots + \gamma^{T-2} r_{T-1}$$

$$\dots$$

$$G_{T-1} = r_1$$

$$G_T = 0$$

3. 테이블 업데이트

	s_1	s_2	s_3
			s_7
s_8	s_9		s_{11}
s_{12}	s_{13}		

→

$(1,-6)$	(0,0)	(0,0)	(0,0)
$(1,-5)$	$(1,-4)$	$(1,-3)$	(0,0)
(0,0)	(0,0)	$(1,-2)$	(0,0)
(0,0)	(0,0)	$(1,-1)$	종료

| 그림 5-5 | 업데이트된 테이블

- $s_0 \to s_4 \to s_5 \to s_6 \to s_{10} \to s_{14} \to end$일때 해당 칸의 N(s)와 V(s)를 다음과 같이 업데이트

$$N(s_t) \leftarrow N(s_t) + 1 V(s_t) \leftarrow V(s_t) + G_t$$

- 모든 상태의 보상은 -1이고 γ는 1

4. 밸류 계산

$$v_\pi(s_t) \cong \frac{V(s_t)}{N(s_t)}$$

조금씩 업데이트하는 버전

$$V(s_t) \leftarrow (1-\alpha) * V(s_t) + \alpha * G_t$$

α=0이면 원래 값 $V(s_t) = 90\%$와 새로운 값 $G_t = 10\%$를 섞음

- 이번에는 에피소드 1개가 끝날 때마다 테이블의 값을 조금씩 업데이트하는 버전

$$V(s_t) \leftarrow V(s_t) + \alpha\big(G_t - V(s_t)\big)$$

- $G_t > V(s_t)$이면 $V(s_t)$를 크게, 그 반대면 작게

몬테 카를로 학습 구현

- 환경: 에이전트의 액션을 받아 상태변이를 일으키고 보상을 줌
- 에이전트: 4방향 랜덤 정책으로 움직임
- 경험 쌓는 부분: 에이전트가 환경과 상호작용하며 데이터를 축적
- 학습하는 부분: 쌓인 경험을 통해 테이블을 업데이트

Gridworld.py

```
# c5/Gridworld.py
import random

class GridWorld:
    # x와 y를 0으로 초기화
    def __init__(self):
        self.x = 0
        self.y = 0

    # 움직임 결정
    # 0: 우, 1: 좌, 2: 상, 3: 하
    def step(self, a):
        if a==0: self.move_right()
        elif a==1: self.move_left()
        elif a==2: self.move_up()
        elif a==3: self.move_down()

        reward = -1
        done = self.is_done()
        return (self.x, self.y), reward, done

    # 움직이는 함수
    def move_right(self):
        self.y += 1
        if self.y > 3: self.y=3

    def move_left(self):
        self.y -= 1
        if self.y < 0: self.y = 0

    def move_up(self):
        self.x -= 1
        if self.x < 0: self.x = 0

    def move_down(self):
        self.x += 1
        if self.x > 3: self.x = 3

    # 완료 여부 함수
    def is_done(self):
        if self.x == 3 and self.y == 3:
            return True
        else: return False

    # 현재 상태
    def get_state(self):
        return (self.x, self.y)

    # 처음으로
    def reset(self):
        self.x = 0; self.y = 0
        return (self.x, self.y)
```

Agent 클래스

```
# c5/Agent.py
import random

class Agent():
    def __init__(self):
        pass

    # 4방향 uniform 랜덤액션 선택
    def select_action(self):
        coin = random.random()
        if coin < 0.25: action = 0
        elif coin < 0.5: action = 1
        elif coin < 0.75: action = 2
        else: action = 3
        return action
```

메인 함수

```
# c5/c5_1_mc.py
from Agent import Agent
from GridWorld import GridWorld

def main():
    env = GridWorld()
    agent = Agent()
    data = [[0]*4 for i in range(4)]
    gamma = 1
    alpha = 0.0001

    for k in range(50000):
        done = False
        history = []
        # 에이전트가 경험을 쌓는 과정
        while not done:
            action = agent.select_action()
            (x, y), reward, done = env.step(action)
            history.append((x, y, reward))
        env.reset()

        # 매 에피소드가 끝나고 바로 해당 데이터를 이용해 테이블 업데이트
        cum_reward = 0
        for transition in history[::-1]:
            x, y, reward = transition
            data[x][y] = data[x][y] + alpha * (cum_reward - data[x][x])
            cum_reward = cum_reward + gamma * reward

    for row in data:
        for i in row:
            print(f'{i:.3f}', end=' ') # 출력창 더러워서 반올림
        print()
```

main()

- while 루프 랜덤 에이전트가 경험을 쌓는 과정
    - while문 내 for 루프는 경험을 업데이트
    - 여기서 리턴 함수 사용

$$G_t = \sum_{i=1}^{\infty} \gamma^{i-1} R_{t+i} = R_{t+1} + \gamma(G_{t+1})$$

학습 결과

$$\begin{pmatrix} -58.639 & 6.981 & 73.138 & 111.507 \\ -142.651 & -53.257 & 45.279 & 109.947 \\ -349.081 & -214.570 & -39.474 & 80.733 \\ -964.002 & -713.365 & -298.718 & 0.000 \end{pmatrix}$$

5.2. Temporal Difference 학습

- 몬테카를로 학습(이하 MC)의 단점: 업데이트를 하려면 에피소드가 끝날 때까지 기다려야 함
    - 즉 종료하는 MDP에서만 사용 가능
- Temporal Difference 학습(이하 TD)에선 에피소드 끝나기 전에 업데이트 가능

이론적 배경

- $v_\pi(s_t) = E_\pi[G - t]$ 에서 "G_t는 $v_p i(s_t)$의 불편 추정량" 이 성립
- 대수의 법칙에 의해 G_t를 여러번 샘플링할수록 $v_\pi(s_t)$로 수렴
- 벨만 방정식 0단계 $v_\pi(s_t) = E_\pi[r_{t+1} + \gamma v_\pi(s_{t+1})]$ 에서 $r_{t+1} + \gamma v_\pi(s_{t+1})$의 기댓값은 $v_\pi(s_t)$.
- 이는 곧 목표지. 즉 TD 타깃
- 값을 한 개씩 얻은 때마다 테이블에 원래 쓰여있던 값을 $r_{t+1} + \gamma v_\pi(s_{t+1})$ 방향으로 조금씩 업데이트

Temporal Difference 학습 알고리즘

- $s_0 \to s_1 \to s_2 \to s_3 \to s_7 \to s_6 \to s_{10} \to s_{11} \to end$ 일때 모든 단계마다 -1의 보상, 8번의 상태전이 → 8개의 상태에 대해 업데이트

$$MC: V(s_t) \leftarrow V(s_t) + \alpha\big(G_t - V(s_t)\big) TD: V(s_t) \\ \leftarrow \alpha\big(r_{t+1} + \gamma V\big(s_{t+1} - V(s_t)\big)\big)$$

- 보상: -1, $\alpha = 0.01$, $\gamma = 1.0$을 사용하면

$$V(s_0) \leftarrow V(s_0) + 0.01 * (-1 + V(s_1) - V(s_0))$$

$$V(s_1) \leftarrow V(s_1) + 0.01 * (-1 + V(s_2) - V(s_1))$$

...

$$V(s_{11}) \leftarrow V(s_{11}) + 0.01 * (-1 + V(s_{15}) - V(s_{11}))$$

메인함수

- GridWorld와 Agent클래스는 위의 몬테 카를로 학습에 쓰던거 그대로 사용

```
from GridWorld import GridWorld
from Agent import Agent

#def main():
env = GridWorld()
agent = Agent()
data = [[0]*4 for i in range(4)]
gamma = 1
alpha = 0.01

for k in range(50000):
    done = False
    while not done:
        x, y = env.get_state()
        action = agent.select_action()
        (x_prime, y_prime), reward, done = env.step(action)
        x_prime, y_prime = env.get_state()

        # 한 번의 스텝이 진행되자마자 바로 테이블의 데이터를 업데이트
        data[x][y] = data[x][y] + alpha * ₩
            (reward + gamma * data[x_prime][y_prime] -
data[x][y])
    env.reset()

for row in data:
    for i in row:
        print(f'{i:.3f}', end=' ') # 출력창 더러워서 반올림
    print()
```

결과

$$\begin{pmatrix} -57.065 & -55.394 & -52.410 & -49.953 \\ -55.030 & -52.342 & -47.841 & -44.252 \\ -52.159 & -47.867 & -39.726 & -29.480 \\ -49.549 & -43.445 & -31.631 & 0.000 \end{pmatrix}$$

5.3. 몬테카를로 vs TD

학습 시점

- MC는 에피소드가 끝나고 리턴이 정해져야 비로소 되돌아보면서 학습을 진행
- TD는 한 스텝이 끝날 때마다 되돌아보면서 학습을 진행

MDP의 종류

- Episode MDP: 종료상태가 존재
- Non-Episode MDP: 종료상태 없이 에피소드가 무한히 이어지는 경우

    - 즉 MC는 Episode MDP에만 적용 가능, TD는 둘 다 적용 가능

편향성(Bias)

- MC는 불편성 만족
- TD의 경우...
    - TD는 $r_{t+1} + \gamma V(s_{t+1})$을 계산하여 TD 타깃으로 삼고, 기댓값 연산자 수식은 $r_{t+1} + \gamma V(s_{t+1})$
- V_π 대신 V를 이용하는 이유
    - $v_\pi(s_{t+1})$을 모음
    - $V_\pi(s_{t+1})$엔 편향 존재
- 즉 TD 타깃은 실제 가치에 다가가리라는 보상 없음
- TD-zero나 뉴럴넷 등의 방법 도입으로 해결

분산

- 이전 테이블에서 MC의 평균 리턴: -60
- 그러나 리턴이 -6이 될수도, -2000이 될수도 있음
    - 분산(혹은 변동성)이 크다는 것을 의미
- TD의 경우 한 걸음을 떼는게 전부라 분산 작음

분산이 학습에서 중요한 이유

- 각각의 샘플은 '오답'이고, 샘플의 평균이 정답
- 정답이 0, 균등분포가 [-10000, 10000] 범위에서 고르는 경우 (정답은 10)
    - 각각의 데이터가 전혀 다른 방향
    - 분산 증가
- 데이터가 [-1, 1]인 경우
    - 확실히 분산 감소

5.4. 몬테 카를로와 TD의 중간?

n 스텝 TD

- TD 타깃: $TD{:}r_{t+1} + \gamma V(s_{t+1})$이 있을 때
    - $s_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 V(s_{t+2})$
- 이를 무한번 반복하면

$$N = \infty{:} \sum_{i=1}^{\infty} \gamma^{i-1} r_{t+1}$$

- 한 극단에는 MC가, 다른 한 극단엔 TD-zero(0스텝 진행)이 있음